

Chapter 9 –Relevance and Query Expansion

<https://chatgpt.com/share/69d9f36e-a620-83e8-82ab-ddb8b0f896be>

<https://drive.google.com/file/d/1vrFNeujz2GooTOFcpMOXPAL1cAePlz3m/view>

<https://drive.google.com/file/d/17OY0cyWDQBZxrVuXOEiEDCpy3FDwU1Rx/view>

<https://drive.google.com/file/d/1lzWkWd7SFx5UnQhmkW2AAN5C0CwMUGwI/view>

★ Problem with IR Systems

One of the biggest challenges in IR systems (like search engines) is **language ambiguity**, especially something called **synonymy**, this has an impact on **recall** !

› Synonymy (same meaning, different words)

Different people may use different words for the same concept.

? Why this is a problem

If a user searches for “**car**”, the system may miss documents that use “**automobile**” instead.

👉 This leads to **low recall**

- **Recall** = how many relevant documents are retrieved out of all relevant documents available

So:

- Relevant docs exist ✓
 - But system doesn't retrieve them ✗
→ Poor recall
-

What do users do?

Users often try to **fix this manually** by:

- **Rewriting queries**
- **Adding more keywords**
- **Trying synonyms** themselves

This is called:

👉 **Query refinement / query expansion**

Query Refinement / Expansion

This is the process of **modifying the user's query** to **improve search results**.

Goal:

- **Retrieve more relevant documents**
- **Improve recall** (and sometimes precision)

There are two major approaches:

- **Global methods**
 - **Local methods**
-

Global Methods

Global methods **improve/expand/reformulate** the query **without looking at the current search results**.

👉 They rely on **external knowledge sources**

Key idea

The system expands or reformulates the query based on general language knowledge.

Common techniques

> Thesaurus or WordNet-based expansion

- Uses a **dictionary of synonyms**
- Example:
 - **Query: "car"**
 - **Expanded to: "car OR automobile OR vehicle"**

This helps match more documents.

> Automatic thesaurus generation

- The system **learns relationships between words automatically**
- Based on:
 - **Co-occurrence in documents**
 - **Statistical similarity**

Example:

- **Words frequently appearing together:**
 - "AI" ↔ "machine learning"
-

> Spelling correction

Fixes user mistakes:

- "recieve" → "receive"

- “enviroment” → “environment”

This ensures:

- Correct matching
 - Better retrieval
-

Key characteristic of global methods

- **Do not depend on search results**
 - Work **before retrieval** or **independently**
-

★ Local Methods

Local methods improve the query **based on** the **documents retrieved from the initial search**.

👉 They are **context-sensitive**

Key idea

- First run the query
 - Look at results
 - Then improve the query using those results
-

Main techniques

› **Relevance Feedback**

This is the most powerful and widely used method.

- The **user gives feedback** on results
 - The **system learns** from that feedback
-

› **Pseudo Relevance Feedback (Blind RF)**

- **Assumes top results are relevant** (even without user input)
- **Automatically expands** query

Example:

- Top 10 results are assumed relevant
 - **Extract important terms from them**
 - **Add those terms** to query
-

› **(global) Indirect relevance feedback**

- Uses **user behavior instead of explicit feedback**
 - Example:
 - **Clicks**
 - **Time spent** on a page
-

Key characteristic of local methods

- **Depend on initial retrieval results**
 - **Adapt to specific query context**
-

★ Relevance Feedback

This is one of the **most effective + most used** techniques in IR.

Core idea >

Involve the **user to teach** the **system** what is **relevant**

Step-by-step process

User submits a query

- Usually short and simple
Example: "machine learning"
-

System retrieves initial results

- Shows a list of documents
-

User gives feedback

Marks documents as:

- Relevant ✓
 - Non-relevant ✗
-

System updates query

- Adds important terms from relevant documents
- Removes or downweights terms from non-relevant ones

System retrieves new results

- Improved and more accurate
-

Why it works

- Captures **true user intent**
 - Reduces ambiguity
 - **Improves both:**
 - **Recall** (more relevant docs found)
 - **Precision** (fewer irrelevant docs)
-

Example

Initial query:

- "apple"

Problem:

- Could mean fruit 🍏 or company 💻

User marks:

- Tech-related docs as relevant

System learns:

- Adds terms like:
 - "iPhone", "Mac", "Apple Inc."

New query becomes:

- “apple iPhone Mac”

Now results are much better 

› Example 2: Initial query/results

■ Initial query: *New space satellite applications*

1. 0.539, 08/13/91, NASA Hasn't Scrapped Imaging Spectrometer
- + 2. 0.533, 07/09/91, NASA Scratches Environment Gear From Satellite Plan
- + 3. 0.528, 04/04/90, Science Panel Backs NASA Satellite Plan, But Urges Launches of Smaller Probes
4. 0.526, 09/09/91, A NASA Satellite Project Accomplishes Incredible Feat: Staying Within Budget
5. 0.525, 07/24/90, Scientist Who Exposed Global Warming Proposes Satellites for Climate Research
6. 0.524, 08/22/90, Report Provides Support for the Critics Of Using Big Satellites to Study Climate
7. 0.516, 04/13/87, Arianespace Receives Satellite Launch Pact From Telesat Canada
8. 0.509, 12/02/87, Telecommunications Tale of Two Companies

+ ■ User then marks relevant documents with “+”.

Query:

👉 “New space satellite applications”

Expanded query after relevance feedback

■ 2.074 new	15.106 space
■ 30.816 satellite	5.660 application
■ 5.991 nasa	5.196 eos
■ 4.196 launch	3.972 aster
■ 3.516 instrument	3.446 arianespace
■ 3.004 bundespost	2.806 ss
■ 2.790 rocket	2.053 scientist
■ 2.003 broadcast	1.172 earth
■ 0.836 oil	0.646 measure

How to Interpret This

This is the **new query vector** (after applying relevance feedback).

Important idea

Each term now has a **weight**

- **Higher weight** = **more important** for the query
- Lower weight = less important

Very high weights

- **satellite (30.816)** → extremely important
- **space (15.106)** → very important

👉 Makes sense because: These appeared frequently in relevant documents

Results for expanded query

1. 0.513, 07/09/91, NASA Scratches Environment Gear From Satellite Plan
2. 0.500, 08/13/91, NASA Hasn't Scrapped Imaging Spectrometer
3. 0.493, 08/07/89, When the Pentagon Launches a Secret Satellite, Space Sleuths Do Some Spy Work of Their Own
4. 0.493, 07/31/89, NASA Uses 'Warm' Superconductors For Fast Circuit
5. 0.492, 12/02/87, Telecommunications Tale of Two Companies
6. 0.491, 07/09/91, Soviets May Adapt Parts of SS-20 Missile For Commercial Use
7. 0.490, 07/12/88, Gaping Gap: Pentagon Lags in Race To Match the Soviets In Rocket Launchers
8. 0.490, 06/14/90, Rescue of Satellite By Space Agency To Cost \$90 Million

Pseudo Relevance Feedback (Blind Feedback)

>a way to **automatically improve a query without asking the user for input.**

Instead of asking the user which documents are relevant, the system **assumes:**

👉 The **top-ranked documents are relevant**

How it works

- User submits a query
 - System retrieves **top k** documents (e.g., top 10)
 - It **assumes these are relevant**
 - **Extracts important terms** from them
 - **Expands the query using those terms**
 - Runs the search again
-

✅ Why it's useful

- **No user effort** required 🙌
- Fully automatic
- Improves results in many cases

⚠️ Limitation

- **Assumption** may be **wrong** ❌
 - If **top results are bad** → **query becomes worse**

👉 This is called **query drift** (**moving away from true intent**)

★ Key takeaway

- It **simulates relevance feedback** without the **user**
 - **Often works** surprisingly well
 - Generally **better than global methods**
-

🧠 Indirect Relevance Feedback (Implicit Feedback)

This method uses **user behavior** instead of explicit input.

💡 Core idea

Users don't tell the system directly what is relevant, but:

👉 Their **actions reveal it**

📊 Examples of implicit signals

- Clicking a result ✓
- Spending more time on a page 🕒

- Scrolling or interacting
 - Skipping certain results
-

How it works

- System **tracks** user **behavior**
 - **Infers** which **documents** are **likely relevant**
 - **Adjusts query** or **ranking accordingly**
-

Advantages

- No effort required from users 🙌
 - **Can collect large-scale data** (especially in search engines)
 - **More realistic** (based on real usage)
-

Limitations

- **Less accurate than explicit** feedback ❌
 - **A click doesn't always mean relevance**
 - **Noisy data** (users may **click randomly**)
-

Important comparison

- **Explicit** feedback → **most accurate** ✓
 - **Implicit** feedback → **scalable & practical** ✓
 - **Pseudo** feedback → **fully automatic** but **risky** ⚠️
-



Relevance Feedback in Vector Space

Model

This explains how relevance feedback is **mathematically applied** using the **vector space model**.

- Represent:
 - **Query** as a **vector**
 - **Documents** as **vectors**
 - Then update the query vector using feedback
-



What happens

- **Only use:**
 - **Relevant documents** ✓
 - Non-relevant documents (optional)
 - **Adjust query:**
 - Move it **closer to relevant documents**
 - Move it **away from non-relevant ones**
-



Effects on performance

- **Improves recall** ✓
(finds more relevant documents)
- **Improves precision** ✓
(reduces irrelevant ones)



When is it most useful?

- Especially **useful** when:
 - **Recall is very important**
 - **User** is **willing** to **iterate and refine**
-

User involvement

- User may:
 - Review results
 - Mark relevance
 - Repeat the process

👉 This **iterative process** leads to **progressively better results**

-  Pseudo feedback → automatic assumption
 -  Indirect feedback → user behavior
 -  **Vector space RF** → **mathematical update of query**
-

Centroid

The **centroid** is the **average (center point)** of a **group of vectors (documents)**.

👉 Think of it like:

- Center of mass 
 - Balance point of multiple points
-

In Information Retrieval

- Each **document** = a **vector** in high-dimensional space

- (Each **dimension** = a **term/word**)

So if you have multiple documents:

👉 Their **centroid = average of all their vectors**

Formula (Centroid)

$$\vec{C} = \frac{1}{|C|} \sum_{\vec{d} \in C} \vec{d}$$

Meaning of formula

- C = set of documents
- $|C|$ = number of documents
- $\vec{d}$ = each document vector

👉 Add all document vectors

👉 Divide by number of documents

= **average vector** (centroid)

- Relevant documents → form a cluster
 - Their centroid = **represents their common meaning**
-

Rocchio Algorithm

> **main algorithm used for relevance feedback** in the vector space model.

Core idea

👉 Improve the query by:

- **Moving it toward relevant documents** ✓
 - **Moving it away from non-relevant documents** ✗
-

Optimal Query (Core Formula)

> following formulas give us the **theoretical best query**:

$$\vec{q}_{opt} = \arg \max_{\vec{q}} [\cos(\vec{q}, \vec{\mu}(C_r)) - \cos(\vec{q}, \vec{\mu}(C_{nr}))]$$

Choose the query vector $\vec{q}$ that **maximizes**:

- ✓ similarity to relevant documents
- ✗ minus similarity to non-relevant documents

◆ $\arg \max_{\vec{q}}$

👉 "Find the query vector $\vec{q}$ such that the expression is maximized"

Think:

- We are **searching for the best possible query**
-

◆ $\cos(\vec{q}, \vec{\mu}(C_r))$

👉 Cosine similarity between:

- Query $\vec{q}$
- Centroid of relevant docs $\vec{\mu}(C_r)$

✓ Measures how close query is to relevant documents

◆ $\cos(\vec{q}, \vec{\mu}(C_{nr}))$

👉 Cosine similarity between:

- Query $\vec{q}$
- Centroid of non-relevant docs

✗ Measures how close query is to irrelevant documents

◆ **The subtraction**

Relevant similarity – Non-relevant similarity

👉 We want:

- First term → **HIGH**
- Second term → **LOW**

$$\vec{Q}_{opt} = \frac{1}{|C_r|} \sum_{\vec{d}_j \in C_r} \vec{d}_j - \frac{1}{|C_{nr}|} \sum_{\vec{d}_j \in C_{nr}} \vec{d}_j$$

$$Q_{opt} = \frac{1}{|C_r|} \sum_{d_j \in C_r} d_j - \frac{1}{N - |C_r|} \sum_{d_j \notin C_r} d_j$$

Q_{opt} = optimal query; C_r = set of rel. doc vectors;
 N = collection size; C_{nr} = set of non-rel. doc

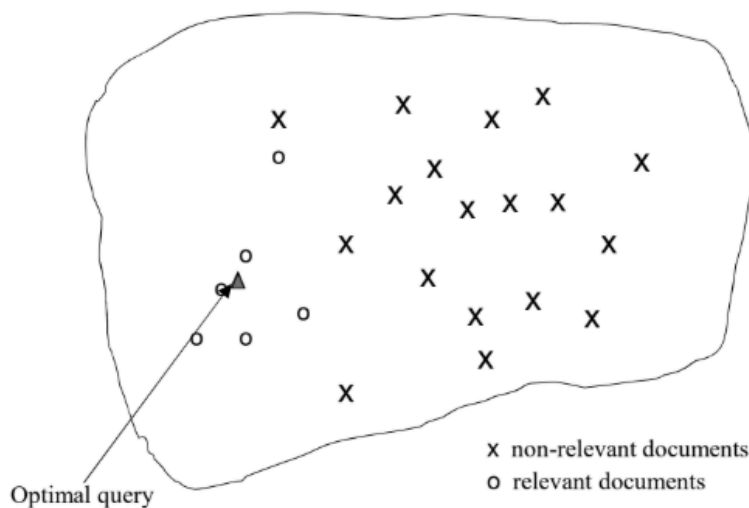
This means that:

👉 The optimal query is:

- Centroid of relevant documents
- minus
- Centroid of non-relevant document

👉 Best query = “Be like relevant docs, avoid non-relevant ones”

The Theoretically Best Query



^ opt. Query towards cluster of relevant docs

⚠ Important Limitation

✗ This is **unrealistic**

Why?

- We don't know:
 - Which docs are relevant
 - Which are non-relevant

👉 That's why we use Rocchio (practical version)

📊 Rocchio Formula (Practical Formula)

$$\vec{q}_m = \alpha \vec{q}_0 + \beta \frac{1}{|D_r|} \sum_{\vec{d}_j \in D_r} \vec{d}_j - \gamma \frac{1}{|D_{nr}|} \sum_{\vec{d}_j \in D_{nr}} \vec{d}_j$$

🧠 Understanding Each Part

◆ $\vec{q}_m$

👉 Modified (new) query vector

✓ This is the **final improved query**

◆ D_r

👉 Set of **relevant documents**

✓ Marked by user (or assumed in pseudo RF)

◆ $\vec{q}_0$

👉 Original query

◆ D_{nr}

👉 Set of **non-relevant documents**

◆ α, β, γ

👉 Control importance:

- α → original query weight
- β → relevant docs influence
- γ → penalty for non-relevant docs

● First term: $\alpha \vec{q}_0$

👉 Keeps original query

- Prevents losing original meaning
- If α is large $\rightarrow$ trust original query more

● Second term: $\beta \frac{1}{|D_r|} \sum \vec{d}_j$

👉 Centroid of relevant documents

✓ Pulls query **toward relevant docs**

✓ Adds useful terms like:

- "NASA"
- "rocket"
- "climate"



● Third term: $\gamma \frac{1}{|D_{nr}|} \sum \vec{d}_j$

👉 Centroid of non-relevant documents

✗ Pushes query **away from irrelevant docs**

✓ Removes noisy terms like:

- "telecommunications"
- "companies"

⚖️ "Tradeoff α vs β/γ "

👉 Important concept:

- If you have **many judged documents**:
 - Increase **β** \ beta and **γ** \ gamma
 - **Trust feedback more**
- If you have **few documents**:
 - Keep **α** **higher**
 - **Trust original query** more

! "Term weight can go negative"

👉 After subtraction:

- Some terms may get **negative weights**

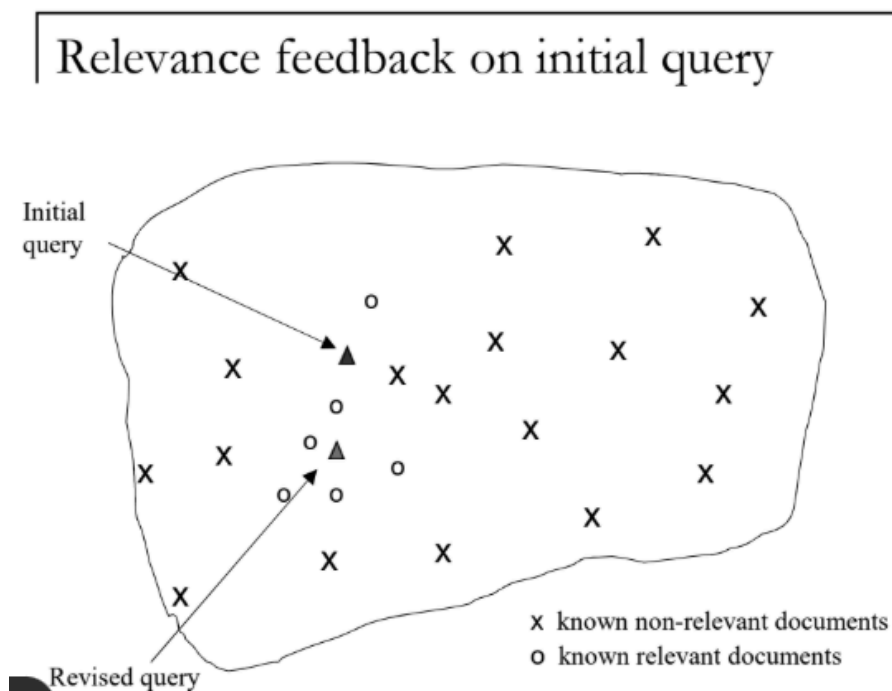
Example: If a term appears mostly in non-relevant docs → negative value

⊘ "Negative term weights are ignored"

👉 In practice:

- We usually:
 - Set **negative weights** to 0 zero
 - Or **remove** those **terms**

✓ Because negative terms don't make sense in queries



Intuition

Imagine a map:

- Query = a point
- Relevant docs = cluster on one side
- Non-relevant docs = cluster elsewhere

👉 Rocchio does this:

- **Pull query toward relevant cluster** 📌
 - **Push** it **away** from **irrelevant** cluster 🚫
-

Relevance Feedback: Assumptions

Relevance feedback (RF) works well **only if certain assumptions** are **true**.

AI: User has sufficient knowledge

👉 Assumption:

- The user **can write** a **reasonable initial query**

✓ That means:

- **Correct spelling**
 - **Relevant keywords**
 - Some idea of the topic
-

A2: Relevance prototypes are “well-behaved”

What is a “prototype”?

 A prototype = **typical representation of relevant documents**

Think: A “**center**” or “**cluster**” of relevant documents

A2 means:

✓ **Relevant documents are similar to each other**

- They use **similar words** / TERM distribution in them is similar
 - Example:
 - “NASA”, “satellite”, “launch”, “orbit”
-

✓ **Non-relevant documents are different**

- They use **different vocabulary**
 - **Term distribution of non-relevant docs is different from relevant docs!!**
 - Example:
 - “telecom”, “business”, “company”
-

Two possible cases

Case 1: Single cluster

 **All relevant documents are:**

- **Close together**
- Easy to identify

✓ **Ideal case for RF**

🟡 Case 2: Multiple clusters (but **overlap** exists)

👉 **Relevant documents** belong to **different groups** BUT:

- **Share some common words**
- **Vocabulary overlap**

✓ Still manageable

! Also assumed:

👉 Relevant vs non-relevant **similarity** is **low**

- Clear separation
 - Easy for system to distinguish
-

⚠️ Violation of A1 (User Problem)

🚫 What happens?

👉 User **does NOT know enough**

📌 Examples

✗ Misspellings

- “Brittany Speers” instead of correct spelling

👉 System may fail to retrieve correct docs

Cross-language issues

- “hígado” (Spanish for liver)

👉 System may not match English documents

Vocabulary mismatch

Mismatch of searcher’s vocabulary vs. collection vocabulary

- User: “cosmonaut”
- Documents: “astronaut”

👉 **Same meaning, different words** → system fails

Result

- **Poor initial results**
 - Relevance feedback becomes **weak** or **misleading**
-

Violation of A2 (Document Problem)

 **What happens?**

👉 **Relevant documents** are **not well-structured**

Case: Multiple unrelated clusters

Example: “Burma / Myanmar”

- Same concept
- Different terms

👉 Documents split into groups

Example: “Contradictory government policies”

- Different viewpoints
 - Different vocabulary
-

Example: “Pop stars that worked at Burger King”

- **Very diverse set:**
 - Different names
 - Different contexts

👉 No consistent vocabulary

🎯 Why this is a problem

👉 Rocchio assumes: One clear centroid

BUT here: **Multiple centroids exist** ❌

👉 **Averaging them** → bad query representation

💥 Result

- Query becomes “confused”
- May include unrelated terms
- Retrieval quality decreases

🧠 Important Insight

✓ When RF works best

- **Clear topic**
- **Similar documents**
- **Consistent vocabulary**

✗ When RF struggles

- **Ambiguous queries**
 - **Multiple meanings**
 - **Diverse document sets**
-



“Good editorial content can help”

👉 Why?

- **Structured articles:**
 - Use consistent terminology
 - Explain concepts clearly

✓ **Helps reduce ambiguity**



Final Summary

- **A1: User must provide a good initial query**
- **A2: Relevant documents must be similar and separable**

Violations:

- ✗ A1 → poor query (spelling, language, vocabulary mismatch)
- ✗ A2 → scattered or inconsistent document clusters



Excite Relevance Feedback (Spink et al., 2000)

This study looked at how people actually use relevance feedback in a real search engine (**Excite**).

Key Findings

Very low usage of relevance feedback

- Only ~4% of users used it
👉 (via “More like this” option)
-

Most users don't explore much

- About 70% users only view first page

👉 They:

- Don't refine queries
 - Don't click feedback options
-

Important insight

👉 That 4% is actually:

- About 1/8 of users who continue searching

✓ So among *active users*, usage is higher

Effectiveness

- RF improved results 2/3 OR ~66% of the time

👉 So:

- It works ✓
- But people don't use it ✗

👉 Relevance feedback is:

- Effective but **underused**
-

Relevance Feedback: Problems

Long queries problem

After RF:

- **Query becomes very long**

Issues

- **Slower response time**
- **Higher computational cost**
- **Inefficient** for IR systems

partial Solution

Use only important terms

- Example:
 - Keep **top 20 terms** by frequency

✓ Reduces complexity

Users don't like giving feedback

- Marking documents is:
 - **Time-consuming**

- Effortful

👉 So users **avoid** it

🤖 Hard to interpret results

After RF:

- **Query changes a lot**
- **Results** become **less transparent**

👉 User thinks: **“Why did I get this result?”**

🧩 Relevance Feedback: Approaches

There are **three** main approaches

- 1) **Term weighting without query expansion**
 - 2) **Query expansion without term weighting**
 - 3) **Query expansion with term weighting**
-

⚖️ Term weighting (no expansion)

👉 Don't add new terms

👉 **Only adjust weights of existing terms**

✓ Safer

✗ Limited improvement

+ Query expansion (no weighting)

👉 Add new **terms**

👉 But **don't assign weights**

✓ Simple

✗ Less precise

🚀 Expansion + weighting (best)

👉 Add new **terms** + **assign importance**

✓ Most powerful

✓ Used in practice

🧪 Noreault (1979) Approach

- User **selects** relevant document(s)
- These **documents** are **treated** as a **new query**
- System **computes similarity** (e.g., cosine similarity) between:
 - **Selected** document(s) and **all other** documents
- Retrieves **top 30 similar documents**
- **Adds these to the original relevant set**
- On average:
 - **~5.1 new relevant documents** found per query

★ Key points

- **Query** itself is **NOT modified**
- Works with **any IR system** (even Boolean)

- Depends on **document similarity**, not query expansion
-

Attar & Fraenkel (1981) Approach

- Uses **all retrieved documents** (not just relevant ones)
- **Extracts terms** from these documents
- **Ranks terms** based on:
 - **Frequency** in retrieved set
 - **Distance** from original query
- **Presents ranked terms** to user
- User can:
 - **Select useful terms** manually
 - Or system tries **automatic selection**

★ **Key points**

- Works even if:
 - **No relevant documents initially found**
 - Humans select good terms ✓
 - Automatic selection performs poorly ✗
 - **Focus** is on **term suggestion**, not documents
-

Dillon et al. (1983) Approach

- Uses **terms from previously retrieved documents**
- Assigns **weights to terms** (similar to term precision)
- Builds a **new Boolean query** automatically
- Does NOT:
 - Rank documents
- Instead:
 - **Improves** the **query structure** itself

★ Key points

- **Designed for** Boolean IR systems
- Combines:
 - **Term weighting + query reformulation**
- Works well only if:
 - **Query construction** is carefully handled

★ Other use of relevance feedback



Following a Changing Information Need

- **User interests change** over time
 - Relevance feedback **updates** the **query accordingly**
 - **Adapts to new terms** (e.g., new products, trends)
 - Keeps **results aligned** with **current user needs**
-



Maintaining an Information Filter

- Used in **news feeds, email filtering, recommendations**
 - User marks items as relevant/irrelevant
 - **System learns user preferences**
 - Automatically **filters future content**
 - **Improves personalization** over time
-



Active Learning

- Used in **machine learning**

- System selects most useful examples to label
- User provides **feedback only on selected data**
- **Reduces annotation cost** and effort
- Improves learning efficiency

+ Query Expansion

- Users give **feedback** on **terms** (**words/phrases**), not documents
 - Identify:
 - Good terms ✓
 - Bad terms ✗
 - **System adds related terms to the query**
 - Helps solve **lexical gap** (different words, same meaning)
 - **Improves recall** (more relevant documents found)
-

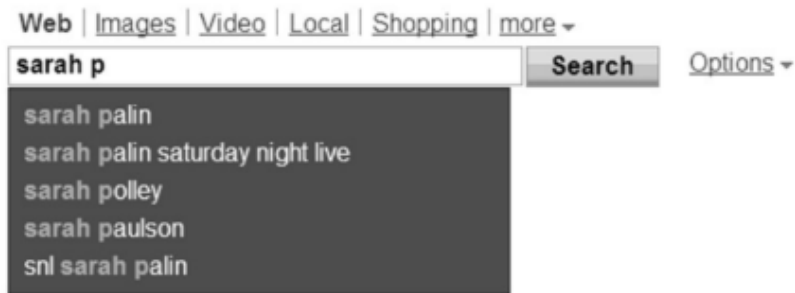
Difference from Relevance Feedback

- **Relevance Feedback:**
 - **Feedback on documents**
 - **Reweights terms**
 - **Query Expansion:**
 - **Feedback on terms**
 - **Adds/removes words**
-

? Query Assist

- **Suggests queries or terms to users**
- Helps users refine search easily

- Example:
 - **Auto-suggestions** / “Did you mean?”



Effect on query volume

- Yes, it **increases query volume** because:
 - Users **try multiple suggested queries**
 - Encourages more interaction
-

How Do We Augment the User Query?

Manual Thesaurus

- **Predefined list of synonyms**
 - Example:
 - physician → doctor, MD, medico
 - Can include: Related terms (not just synonyms)
 - Accurate but:
 - **Limited coverage** + **Requires manual effort**
-

Global Analysis (Static)

- **Uses entire document collection**
 - Methods:
 - **Automatically generated thesaurus**
 - **Query log mining** (analyzing past searches)
 - Characteristics:
 - Independent of current query
 - Works on overall data
-

⚡ Local Analysis (Dynamic)

- Uses **top retrieved documents**
- **Extracts important terms** from them
- **Expands query based on** these **terms**
- Characteristics:
 - **Query-specific**
 - **More context-aware**

◀Food for Thoughts▶

1. What do we mean by relevance feedback? What is the procedure?

Relevance feedback is a technique where the system improves search results using **user judgments** on **retrieved documents**.

Procedure: user submits query → system retrieves results → user marks relevant/non-relevant → system updates query → new results shown.

2. Give three reasons why relevance feedback has been little used in web search.

- Users are **reluctant** to **give explicit feedback**
- Most users **only view first page** of results

- Feedback process **increases effort** and **time**
-

3. What do we mean by Pseudo relevance feedback? Give example.

Pseudo relevance feedback **assumes top retrieved documents** are **relevant** and **uses them** to **expand** the **query automatically**. Example: top 10 results for “AI” are assumed relevant and used to add terms like “machine learning”.

4. Rocchio calculation:

Suppose that a user’s initial query is “**cheap CDs cheap DVDs extremely cheap CDs**”. The user examines two documents, d1 and d2.

She judges d1, with the content “**CDs cheap software cheap CDs**” relevant and d2 with content “**cheap thrills DVDs**” non relevant.

Assume that we are using **direct term frequency** (with no scaling and no document frequency). There is no need to length-normalize vectors.

Using Rocchio relevance feedback, what would the revised query vector be after relevance feedback? Assume $\alpha = 1$, $\beta = 0.75$, $\gamma = 0.25$.

<https://chatgpt.com/share/6a0cb114-15bc-8323-8497-533c9f196152> **[vector sol]**

We are using the Rocchio formula:

$$\vec{Q}_{new} = \alpha \vec{Q}_{old} + \beta \vec{d}_r - \gamma \vec{d}_{nr}$$

(Simplified version since there is only one relevant doc (d_1) and one non-relevant doc (d_2))

👉 You must consider **ALL** terms that appear in:

- Query
- Relevant docs
- Non-relevant docs

Initial query:

cheap(3), CDs(2), DVDs(1), extremely(1)

d1 (relevant):

CDs(2), cheap(2), software(1), relevant(1)

d2 (non-relevant):

cheap(1), thrills(1), DVDs(1)

Now apply:

$$Q_m = Q_0 + 0.75 * d1 - 0.25 * d2$$

Final vector:

- cheap = $3 + (0.75 \times 2) - (0.25 \times 1) = \mathbf{4.25}$
- CDs = $2 + (0.75 \times 2) = \mathbf{3.5}$
- DVDs = $1 - (0.25 \times 1) = \mathbf{0.75}$
- extremely = **1**
- software = 0.75
- thrills = -0.25 (ignored in practice)

< Food for Thoughts > Q4.

initial Q: cheap(3), CDs(2), DVDs(1), extremely(1)

• d1: CDs(2), cheap(2), software(1) → marked relevant

• d2: cheap(1), Thrills(1), DVDs(1) → marked non-rel.

$\alpha = 1, \beta = 0.75, \gamma = 0.25$

now for each term:-

$$Q_{\text{new}} = \alpha Q_0 + \beta d1 - \gamma d2$$

* cheap = $3(1) + (0.75)(2) - (0.25)(1) = 4.25$ → words w weights!

* CDs = $(1)(2) + (0.75)(2) - 0 = 3.5$

* DVDs = $(1)(1) + 0 - (0.25)(1) = 0.75$

* extremely = 1

* software = $0 + (0.75)(1) - 0 = 0.75$

* Thrills = -0.25 → we ignore it, cuz its -ve.

* must consider All terms that appear in query, relevant n non-rel docs.

5. What is a query expansion? Why it is a costly phenomena?

Query expansion **adds related terms** to improve retrieval. It is costly because it **increases query size, computation time, and retrieval cost.**

6. What is a general approach to give a global relevance feedback? Give an example.

Global relevance feedback uses **external resources independent** of **current results**.
Example: using a **thesaurus** to expand "doctor" into "physician, medic".

7. Is negative feedback of any importance in web-search? Explain.

Yes, it **helps remove irrelevant terms** and **improves precision**, but in practice it is less used because users **rarely provide negative feedback.**

8. What are some of the problems of search engine on which relevance feedback effective? Outline them.

- **Synonymy** (different words, same meaning)
 - **Vocabulary mismatch**
 - **Ambiguous queries**
 - Low recall problems
-

9. What are the assumptions for Rocchio algorithms for relevance feedback? Why these assumptions are not correct? Explain.

Assumptions: user gives **good initial query** (A1) and **relevant documents** form **clear clusters** (A2). These are often incorrect because users may lack knowledge and relevant documents may be diverse or scattered.

➤ EXERCISES:

Problem 9.1

- In Rocchio's algorithm, what weight setting for $\alpha/\beta/\gamma$ does a "Find pages like this one" search correspond to?
 - "Find pages like this one" ignores the query and no negative judgments are used.
 - A good positive judgement is available (Example Page)
 - Hence $\alpha = \gamma = 0$. This implies $\beta = 1$.

👉 "Find pages like this one" means:

- Treat the given page as the **entire query**
- Just find **similar documents**

Problem 9.3

- Give three reasons why relevance feedback has been little used in web search?
 - RF slows down returning results as you need to run two sequential queries, the second of which is slower to compute than the first. Web users hate to be kept waiting.
 - RF is mainly used to increase recall, but web users are mainly concerned about the precision of the top few results.
 - RF is one way of dealing with alternate ways to express an idea (synonymy), but indexing anchor text is commonly already a good way to solve this problem.
 - RF complicates the user interface.
 - RF is difficult to explain.

Problem 9.b

- What are some of the problems associated with relevance feedback?
 - Relevance feedback is expensive
 - Relevance feedback creates long modified queries
 - Long queries are expensive to process
 - Users are reluctant to provide explicit feedback
 - Its often hard to understand why a particular document was retrieved after applying relevance feedback

Problem 9.c

- Discuss the pros and cons of implicit (indirect) vs. explicit (direct) feedbacks ?

Implicit(Indirect) feedback	Explicit (Direct) feedback
Implicit (indirect) feedback does not bother user for explicit actions. It is fast and can be possible for large IR system. It is less reliable and possibly introduce a problem of query drift.	Explicit (Direct) feedback requires user to marks document relevant. It is slow process and does not scale to large systems. It is more reliable and generally save from query drift.

Problem 9.d

- What is meant by query expansion? Give an example query that need expansion? When query expansion is very useful?
 - Query expansion is an autonomous process of reformulating a seed query (q_0) to improve retrieval performance in information retrieval systems.
 - It is generally performed to bridge the gap between user information need and the posed query.
 - A general case is a lexical mismatch for example astronauts or cosmonauts are mean the same thing, the query for astronauts implicitly union with the term cosmonauts to bridge the gap and get the relevant documents from both the terms.
 - It is very useful technique for such a situation.

Problem

Suppose that a user's initial query is $q = w_1 w_3 w_2$ and IR systems return four documents. User selected $d_1 = w_2 w_3 w_4$, $d_2 = w_3 w_4 w_5$ and $d_4 = w_1 w_3 w_4 w_1$ as relevant. While $d_3 = w_2 w_4 w_5 w_3$ as non-relevant to her query. Assume that we are using direct term frequency (with no scaling and no document frequency). There is no need to length-normalize vectors. Using Rocchio relevance feedback algorithm to get modify query vector (optimal) after relevance feedback? Rocchio equation is given below.

$$\vec{q}_m = \alpha \vec{q}_0 + \beta \frac{1}{|D_r|} \sum_{\vec{d}_j \in D_r} \vec{d}_j - \gamma \frac{1}{|D_{nr}|} \sum_{\vec{d}_j \in D_{nr}} \vec{d}_j$$

$$q = \langle 1, 1, 1, 0, 0 \rangle$$

$$d_1 = \langle 0, 1, 1, 1, 0 \rangle$$

$$d_2 = \langle 0, 0, 2, 1, 1 \rangle$$

$$d_3 = \langle 0, 1, 1, 1, 1 \rangle$$

$$d_4 = \langle 2, 0, 1, 1, 0 \rangle$$

>

Let's $\alpha=0.1$, $\beta=0.6$, and $\gamma=0.3$

[Problem] [pdf] [slide 5-6]

$$\alpha = 0.1, \beta = 0.6, \gamma = 0.3$$

initial query $\rightarrow q_0 = \langle 1, 1, 1, 0, 0 \rangle$

$\rightarrow d_1 = \langle 0, 1, 1, 1, 0 \rangle$, $d_2 = \langle 0, 0, 2, 1, 1 \rangle$

$d_3 = \langle 0, 1, 1, 1, 1 \rangle \rightarrow nr$

$d_4 = \langle 2, 0, 1, 1, 0 \rangle \rightarrow r$

• set of relevant docs (D_r): d_1, d_2, d_4

$$|D_r| = 3$$

• set of non-rel docs: (D_{nr}): d_3

$$|D_{nr}| = 1$$

$$\ast \vec{q}_m = \alpha \vec{q}_0 + \beta \frac{1}{|D_r|} \sum_{\vec{d}_j \in D_r} \vec{d}_j - \gamma \frac{1}{|D_{nr}|} \sum_{\vec{d}_j \in D_{nr}} \vec{d}_j \quad - (1)$$

• first, calc centroid of Relevant docs;

$$\frac{1}{|D_r|} \sum_{d_j \in D_r} \vec{d}_j = \frac{1}{3} \langle 0+0+2, 1+0+0, 1+2+1, 1+1+1, 0+1+0 \rangle$$

$$= \frac{1}{3} \langle 2, 1, 4, 3, 1 \rangle = \langle 0.667, 0.333, 1.333, 1, 0.333 \rangle$$

• now calc centroid of non-relevant docs;

$$\frac{1}{|D_{nr}|} \sum_{d_j \in D_{nr}} \vec{d}_j = \frac{1}{1} \langle 0, 1, 1, 1, 1 \rangle = \langle 0, 1, 1, 1, 1 \rangle$$

Put it in eq ①;

$$\vec{q}_m = (0.1) \langle 1, 1, 1, 0, 0 \rangle + (0.6) \langle 0.667, 0.333, 1.333, 1, 0.333 \rangle - (0.3) \langle 0, 1, 1, 1, 1 \rangle$$

$$= \langle 0.1, 0.1, 0.1, 0, 0 \rangle + \langle 0.4, 0.2, 0.8, 0.6, 0.2 \rangle - \langle 0, 0.3, 0.3, 0.3, 0.3 \rangle$$

$$\vec{q}_m = \langle 0.5, 0, 0.6, 0.3, -0.1 \rangle$$

› what if the question said that we have to length normalize the vectors, what would we have done then?

Instead of using the raw term frequencies (the counts), you would scale every vector so that its "length" (magnitude) equals 1.

1. Calculate the Magnitude (Length) of each Vector

For any vector $v = \langle x_1, x_2, \dots, x_n \rangle$, the length $\|v\|$ is calculated using the Euclidean formula:

$$\|v\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

2. Create the Normalized Vectors

You divide every component of the vector by its length to get the unit vector $\hat{v}$:

$$\hat{v} = \frac{v}{\|v\|}$$

Example: Normalizing d_1 from your problem

Original $d_1 = \langle 0, 1, 1, 1, 0 \rangle$

1. Find the length:

$$\|d_1\| = \sqrt{0^2 + 1^2 + 1^2 + 1^2 + 0^2} = \sqrt{3} \approx 1.732$$

2. Normalize it:

$$\hat{d}_1 = \langle \frac{0}{1.732}, \frac{1}{1.732}, \frac{1}{1.732}, \frac{1}{1.732}, \frac{0}{1.732} \rangle \approx \langle 0, 0.577, 0.577, 0.577, 0 \rangle$$

Then we wouldve applied the **Rocchio formula** to these normalized vectors!